

Database Management Systems

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Database Management Systems - Complete Notes	4
1. Introduction to Database Systems	4
1.1 Basic Concepts	4
1.2 Database Architecture	5
1.3 Database Users and DBA	6
2. Data Models	7
2.1 Database Models	7
2.2 Data-Structure Diagrams (Network Model)	8
3. Entity-Relationship (ER) Model	8
3.1 ER Model Components	8
3.2 Relationships	9
3.3 Keys	10
3.4 ER Diagram (ERD) Notations	10
3.5 Extended ER (EER) Features	10
3.6 Reduction of ER Diagram to Relational Tables	10
4. Relational Model	11
4.1 Relational Concepts	11
4.2 Integrity Constraints	12
5. Relational Algebra and Calculus	12
5.1 Relational Algebra (Procedural)	12
5.2 Relational Calculus (Non-Procedural)	13
6. Structured Query Language (SQL)	14
6.1 SQL Basics	14
6.2 DDL - Data Definition Language	14
6.3 DML - Data Manipulation Language	15
6.4 DQL - Querying Data (SELECT)	15
6.5 Joins in SQL	15
6.6 Advanced Queries	15
6.7 DCL and TCL	16
6.8 PL/SQL and Active Database	17
7. Relational Database Design and Normalization	17
7.1 Functional Dependencies (FD)	17
7.2 Finding Candidate Keys from FDs	18
7.3 Decomposition	18
7.4 Normal Forms	18
8. Transaction Management and Concurrency Control	19

8.1 Transaction Concepts	19
8.2 Schedules	20
8.3 Concurrency Control Protocols	21
9. Recovery and File / Storage Structure	22
9.1 Database Recovery	22
9.2 File Organization and Indexing	23
10. Advanced and Emerging Topics (Overview)	24
10.1 Distributed Databases	24
10.2 Data Warehousing and OLAP	25
10.3 NoSQL Databases	25
10.4 Query Processing and Optimization	25
11. Quick Revision / Exam Pointers	25

Database Management Systems - Complete Notes

A **Database Management System (DBMS)** is system software for **creating, storing, retrieving, updating and managing** data in a database. This guide deconstructs the entire IBPS SO (IT Officer) DBMS syllabus to the atomic level, progressively building from first principles to advanced transaction theory and indexing.

Memory Trick: The 4 jobs of any DBMS = **CRUD** = **C**reate, **R**ead, **U**ppdate, **D**elete.

1. Introduction to Database Systems

1.1 Basic Concepts

1.1.1 Data, Information, Database, DBMS

- **Data:** raw, unprocessed, known facts having no meaning by themselves (e.g. 42, "Ram").
- **Information:** processed, organized data that is meaningful and useful for decision-making.
- **Knowledge:** information combined with experience/context used to act.
- **Database:** an **organized, shared, persistent** collection of inter-related data.
- **DBMS:** the **software layer** between the physical database and the user/application that controls all access.
- **Metadata:** "data about data" - definitions of tables, columns, types, stored in the **data dictionary**.

The chain is: **Data -> Information -> Knowledge**. A DBMS plus the data it manages is called a **Database System**.

MCQ Trap: Data is the raw fact; **information** is processed data. Examiners swap these definitions. The **DBMS is the software**, the **database is the data** - they are not the same thing.

1.1.2 File Management System vs DBMS

A traditional **file-processing system** stores data in separate OS files, each application owning its own files. This causes **data redundancy** and **inconsistency**.

Feature	File System	DBMS
Data redundancy	High (duplicate files)	Controlled / minimal
Data consistency	Poor (anomalies)	Enforced via constraints
Data abstraction	None	3 levels of abstraction
Concurrency control	Absent / manual	Built-in (locks, timestamps)
Security	File-level only	Fine-grained (GRANT/REVOKE)
Query facility	None (write code)	Declarative (SQL)
Integrity constraints	Hard to enforce	Easy (PK, FK, CHECK)
Recovery / backup	Manual	Automatic (logs, checkpoints)
Data independence	Absent	Physical + Logical
Cost / complexity	Low	High

Memory Trick: DBMS beats files on "**R-I-C-S**" = **R**edundancy control, **I**ntegrity, **C**oncurrency, **S**ecurity.

1.1.3 Advantages of DBMS

- **Controlled redundancy** and elimination of **inconsistency**.
- **Data sharing** among multiple users/applications concurrently.
- **Data integrity** through constraints.
- **Data security** via authorization (GRANT/REVOKE).

- **Backup and recovery** subsystems.
- **Data independence** (physical and logical).
- **Enforcement of standards** and reduced application-development time.
- **Atomicity** of updates via transactions.

1.1.4 Disadvantages / Limitations of DBMS

- **High cost** of software, hardware and trained staff.
- **Complexity** of design and administration.
- **Performance overhead** - generality costs speed for simple single-user tasks.
- **Larger size / memory footprint.**
- **Single point of failure** - if the DB server fails all applications stop.

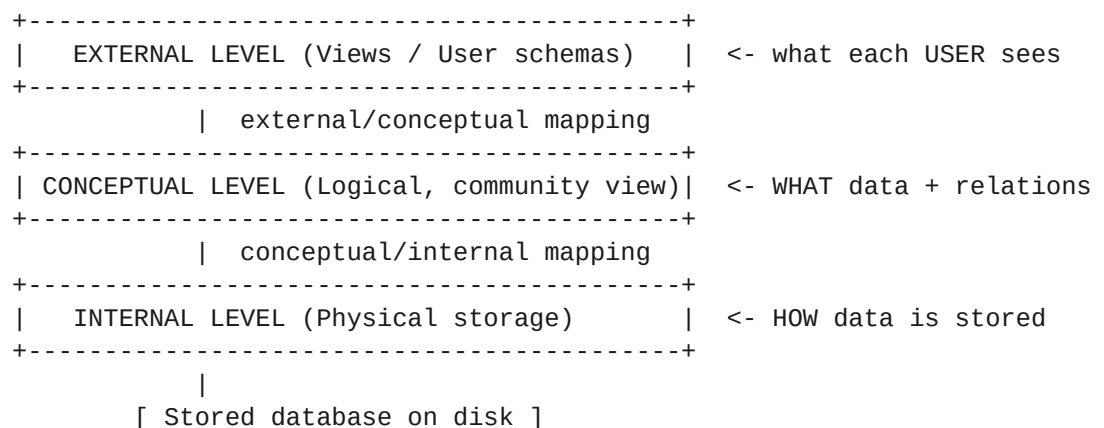
1.1.5 Applications of DBMS

- **Banking, Airlines/Railways reservation, Universities, E-commerce, Telecom, HR/Payroll, Libraries, Social media, Healthcare.**

1.2 Database Architecture

1.2.1 Three-Schema (ANSI/SPARC) Architecture

Proposed by **ANSI/SPARC in 1975**, it separates the database into **3 levels** to provide **data independence**.



- **Internal / Physical level:** lowest level; describes **how** data is physically stored (files, indexes, B+ trees, compression). Defined by the **internal schema**.
- **Conceptual / Logical level:** describes **what** data is stored and the relationships; one **conceptual schema** for the whole DB; used by the DBA.
- **External / View level:** highest level; each user group sees only a **subset** via **views**; hides the rest for **security + simplicity**. There can be **many external schemas**.

Level	Schema	Describes	Audience
External	View / sub-schema (many)	User-specific portion	End users / apps
Conceptual	Conceptual schema (one)	Entities, attributes, relations	DBA / designers
Internal	Internal schema (one)	Storage, files, indexes	System / storage

MCQ Trap: Internal = physical = HOW; Conceptual = logical = WHAT; External = view = WHO sees what. There is exactly **one conceptual** and **one internal** schema but **many external** schemas.

1.2.2 Data Independence

Data independence = capacity to change the schema at one level without altering the schema at the next higher level.

- **Physical Data Independence:** change the **internal** schema (storage, indexes) without changing the **conceptual** schema. **Easier to achieve.**
- **Logical Data Independence:** change the **conceptual** schema (add a column/entity) without changing **external** schemas/applications. **Harder to achieve.**

Memory Trick: "Physical is Possible easily; Logical is Laborious (hard)."

1.2.3 Data Abstraction

Hiding low-level complexity from users; realized by the **3 levels** (physical, logical, view). The DBMS exposes only relevant data.

1.2.4 Instances and Schemas

- **Schema:** the **overall design / structure** of the database; changes **rarely**; the "intension". Types: physical, logical, view schema.
- **Instance (State):** the **data in the database at a particular moment**; changes **frequently**; the "extension".

MCQ Trap: Schema = blueprint (stable); Instance = current data snapshot (volatile). Schema is to instance what a class is to an object.

1.3 Database Users and DBA

1.3.1 Database Administrator (DBA)

The **DBA** has central control of the data and programs. Responsibilities:

- **Schema definition** and storage-structure/access-method definition.
- **Granting authorization** (security) via GRANT/REVOKE.
- **Routine maintenance:** backups, performance tuning, free-space management.
- **Integrity-constraint** specification.

1.3.2 Types of Database Users

User type	Interaction
Naive / Parametric users	Use ready-made forms/apps (e.g. bank teller)
Application programmers	Write programs using embedded/host SQL
Sophisticated users	Write ad-hoc queries directly (analysts)
Specialized users	Build CAD, expert, scientific DB apps
DBA	Administers, controls and tunes the DB

1.3.3 Database Languages (Sub-languages of SQL)

Language	Full form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify schema
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Modify data
DQL	Data Query Language	SELECT	Retrieve data
DCL	Data Control Language	GRANT, REVOKE	Permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

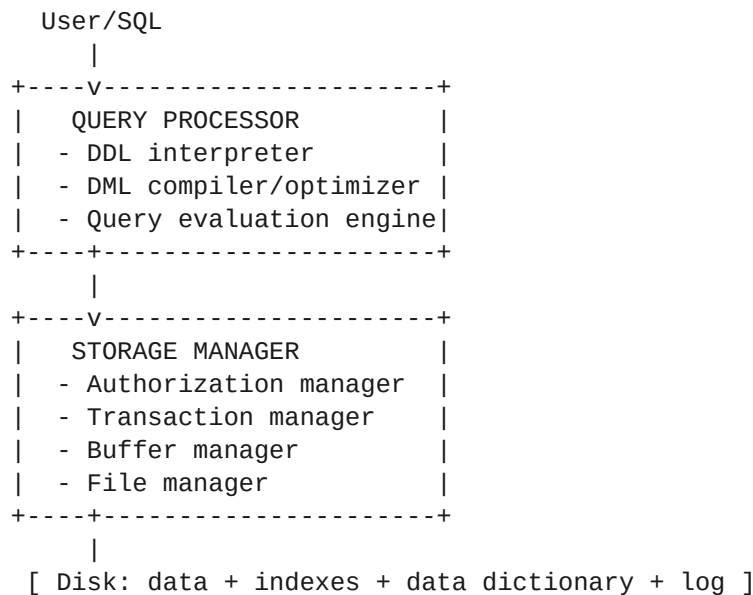
- **DDL** output is stored in the **data dictionary**.
- **DML** is of **2 types**: **procedural** (how + what, e.g. relational algebra) and **non-procedural / declarative** (only what, e.g. SQL, relational calculus).

Memory Trick: "Dr. DC T" - **DDL**, **DML**, **DQL**, **DCL**, **TCL**. TRUNCATE and DROP are **DDL** (auto-commit), DELETE is **DML** (can rollback).

1.3.4 Data Dictionary / System Catalog

A **system catalog** storing **metadata**: table/column names, types, constraints, indexes, users, privileges, statistics. It is **self-describing**; the DBMS itself reads it during query processing.

1.3.5 Database Engine Components



- **Query processor**: DDL interpreter, **DML compiler + query optimizer**, query evaluation engine.
- **Storage manager**: **authorization & integrity manager**, **transaction manager**, **buffer manager**, **file manager**.
- **Buffer manager** is critical - it decides what data stays in main memory.

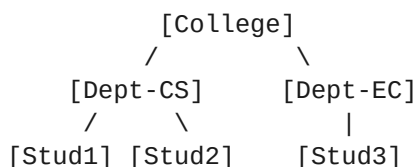
2. Data Models

A **data model** is a collection of conceptual tools for describing **data, relationships, semantics and constraints**. Categories: **high-level/conceptual** (ER), **representational/implementation** (relational, network, hierarchical), **low-level/physical**.

2.1 Database Models

2.1.1 Hierarchical Model

- Data organized as a **tree** (inverted); each child has **exactly one parent (1:N only)**.
- First used in **IBM IMS (1968)**.
- Relationship is **parent-child (PCR)**. Fast for 1:N; **cannot model M:N** directly; rigid.



2.1.2 Network Model

- Data as a **graph**; a child (member) can have **multiple parents (owners)** - supports **M:N**.
- Standardized by **CODASYL DBTG**. Uses **set** construct (owner-member).
- More flexible than hierarchical but **complex pointers** to navigate.

2.1.3 Relational Model

- Proposed by **E. F. Codd (IBM) in 1970**.
- Data stored as **relations (two-dimensional tables)** of rows (tuples) and columns (attributes).
- Simplicity, strong theory (set theory + predicate logic), data independence**. Basis of SQL DBs (Oracle, MySQL, PostgreSQL, SQL Server).

2.1.4 Object-Oriented Model

- Data as **objects** (like OOP) with **attributes + methods**, identity (OID), inheritance, encapsulation.
- Suited to complex data (CAD, multimedia). Example: **ObjectStore, db4o**.

2.1.5 Object-Relational Model

- Hybrid**: relational core extended with OO features (user-defined types, inheritance, nested tables). Example: **PostgreSQL, Oracle**.

2.1.6 Entity-Relationship Model

- A **high-level conceptual** model (Peter Chen, **1976**) using **entities, attributes, relationships**; drawn as an **ER diagram**; later mapped to relational tables.

Model	Structure	Relationship	M:N?
Hierarchical	Tree	Parent-child	No
Network	Graph	Owner-member set	Yes
Relational	Tables	Foreign keys	Yes
ER	Diagram	Relationship sets	Yes (conceptual)

MCQ Trap: Hierarchical = tree (1:N, single parent); Network = graph (M:N, multiple parents). Relational model was given by **Codd (1970)**; ER by **Chen (1976)**.

2.2 Data-Structure Diagrams (Network Model)

- A **data-structure diagram** depicts the schema of a **network** database: **records** as boxes and **links (sets)** as arrows.
- DBTG set concept**: a set type has **one owner** record type and **one or more member** record types; an occurrence has exactly one owner and zero-or-more members; used to model **1:N** links explicitly. **M:N** must be broken into two 1:N sets via a dummy record.

3. Entity-Relationship (ER) Model

3.1 ER Model Components

3.1.1 Entity, Entity Set, Entity Type

- Entity**: a real-world **object/thing** distinguishable from others (e.g. a student **Ram**).
- Entity Type**: the **definition/structure** (e.g. STUDENT with attributes roll, name).
- Entity Set**: the **collection of all entities** of one type at a time (like a table). Drawn as a **rectangle**.

3.1.2 Strong vs Weak Entity

- Strong entity**: has its **own primary key**; drawn as a **single rectangle**.
- Weak entity**: has **no key of its own**; depends on a **strong (owner/identifying) entity**; identified by a **partial key + owner's key**. Drawn as a **double rectangle**; its identifying relationship is a **double diamond**.

[Strong]====<< depends >>====[[Weak]]
 (single) (double diamond) (double)

MCQ Trap: A **weak entity** always has **total participation** in its identifying relationship and uses a **double rectangle**; its **partial key** is underlined with a **dashed line**.

3.1.3 Attributes

Attribute type	Meaning	ERD symbol
Simple (atomic)	Cannot be divided	Oval
Composite	Divisible (Name -> First, Last)	Oval with sub-ovals
Single-valued	One value (Age)	Oval
Multi-valued	Many values (PhoneNo)	Double oval
Stored	Physically stored (DOB)	Oval
Derived	Computed (Age from DOB)	Dashed oval
Key attribute	Uniquely identifies	Underlined label
Null	Unknown / not applicable	-

Memory Trick: "Double = many" - double oval = multivalued, double rectangle = weak entity, double diamond = identifying relationship, dashed oval = derived.

3.2 Relationships

- **Relationship:** an **association** among entities (e.g. STUDENT *enrolls* COURSE). Drawn as a **diamond**.
- **Relationship set:** set of relationships of the same type.
- **Degree:** number of entity types participating: **Unary (1) / Recursive, Binary (2), Ternary (3), n-ary.**

3.2.1 Mapping Cardinality

The number of entities to which another entity can be associated. **4 types:**

- **One-to-One (1:1):** one A relates to one B.
- **One-to-Many (1:N):** one A relates to many B.
- **Many-to-One (M:1):** many A relate to one B.
- **Many-to-Many (M:N):** many A relate to many B.

1:1 A ---- B 1:N A ---< B, B, B
 M:1 A, A, A >---- B M:N A, A >--< B, B

3.2.2 Participation Constraints

- **Total participation (mandatory):** **every** entity must participate; drawn with a **double line**.
- **Partial participation (optional):** **some** entities participate; **single line**.

3.2.3 Recursive (Unary) Relationship

- An entity set related to **itself** (e.g. EMPLOYEE *manages* EMPLOYEE). Uses **role names** to distinguish.

MCQ Trap: Degree of a relationship = number of entity sets it connects (not the cardinality). **Cardinality ratio** is 1:1/1:N/M:N. Don't confuse degree with cardinality.

3.3 Keys

Key	Definition
Super Key	Any attribute set that uniquely identifies a tuple (may have extra attributes)
Candidate Key	A minimal super key (no proper subset is a super key)
Primary Key	The one chosen candidate key; NOT NULL + UNIQUE
Alternate Key	Candidate keys not chosen as primary
Foreign Key	Attribute referring to primary key of another (or same) relation
Composite Key	A key made of 2 or more attributes
Surrogate Key	System-generated artificial key (e.g. auto-id)

Relationship: **Primary Key (subset of) Candidate Key (subset of) Super Key.**

Memory Trick: "Super -> Candidate -> Primary" = going from **largest** set to the **single chosen minimal** key. Every primary key is a candidate key; every candidate key is a super key; the reverse is not true.

3.4 ER Diagram (ERD) Notations

[Rectangle]	= Entity set	([[double]]) = weak
(Oval)	= Attribute	((double)) = multivalued
<Diamond>	= Relationship	<<double>> = identifying
___Underline___	= Primary key	----line---- = single/partial
====double line=	= Total participation	

3.5 Extended ER (EER) Features

3.5.1 Specialization and Generalization

- **Specialization (top-down):** split a **higher entity** into **specialized sub-entities** based on distinguishing features (EMPLOYEE -> ENGINEER, MANAGER).
- **Generalization (bottom-up):** combine **several entities** into a **generalized super-entity** (CAR, TRUCK -> VEHICLE).
- Represented by an **ISA triangle**; supports **attribute inheritance**.

Constraint	Options
Disjointness	Disjoint (one subclass only) vs Overlapping
Completeness	Total (must belong to a subclass) vs Partial

3.5.2 Aggregation

- Treats a **relationship (with its entities) as a higher-level entity** so it can participate in another relationship. Solves the limitation that ER cannot draw a relationship **between an entity and a relationship**.

3.5.3 Categorization / Union Type

- A subclass whose members are a **union of entities from different superclasses** (e.g. OWNER = PERSON UNION COMPANY). Drawn with a **circle + "U"**.

MCQ Trap: Specialization = top-down (divide); Generalization = bottom-up (combine). Aggregation = relationship treated as an entity; do not confuse it with generalization.

3.6 Reduction of ER Diagram to Relational Tables

Rules to map an ER schema to tables:

- **Strong entity** -> a table with all simple attributes; PK = entity key.
- **Weak entity** -> a table including the **owner's PK as FK**; PK = owner PK + partial key.

- **1:1 relationship** -> add FK on either side (prefer total-participation side).
- **1:N relationship** -> add the **PK of the "1" side as FK** in the "N" side table (no separate table needed).
- **M:N relationship** -> create a **separate (junction) table** with **PKs of both sides as a composite key**.
- **Multivalued attribute** -> a **separate table** (owner PK + the attribute).
- **Composite attribute** -> store only the **simple sub-parts** as columns.

Memory Trick: "M:N always makes a new table; 1:N just borrows a foreign key."

4. Relational Model

4.1 Relational Concepts

- **Relation:** a **table** (a set of tuples); mathematically a **subset of the Cartesian product** of domains.
- **Tuple:** a **row** of the table.
- **Attribute:** a **column** (named).
- **Domain:** the **set of permitted atomic values** for an attribute.
- **Relation schema:** $R(A_1, A_2, \dots, A_n)$ - the structure (name + attributes).
- **Relation instance:** the **set of tuples** present at a moment.

4.1.1 Degree and Cardinality

- **Degree (arity):** number of **attributes (columns)** in the relation.
- **Cardinality:** number of **tuples (rows)** in the relation instance.

Degree = 3 (columns)				
+-----+	+-----+	+-----+	+-----+	
RollNo	Name	Branch		<- attributes
+-----+	+-----+	+-----+	+-----+	
101	Ram	CSE	\	
102	Sita	ECE	}	Cardinality = 3 (rows)
103	Ravi	CSE	/	
+-----+	+-----+	+-----+	+-----+	

4.1.2 Properties of Relations

- Each **tuple is unique** (no duplicate rows - it is a **set**).
- **Tuple order is immaterial; attribute order is immaterial.**
- Each cell holds **one atomic value** (no multivalued cells).
- All values in a column come from the **same domain**.
- **Attribute names are distinct** within a relation.

MCQ Trap: Degree = number of columns/attributes; Cardinality = number of rows/tuples. Examiners flip these constantly. A relation with no tuples still has its degree.

4.2 Integrity Constraints

Constraint	Rule
Domain constraint	Each value must belong to the attribute's domain/type
Key constraint	Candidate key values must be unique (no two tuples same key)
Entity integrity	Primary key cannot be NULL
Referential integrity	A foreign key must match an existing PK value or be NULL
NOT NULL	Column cannot store NULL
UNIQUE	All values distinct (NULLs allowed, usually one)
CHECK	Value must satisfy a boolean condition
DEFAULT	Supplies a value when none given

- Referential integrity** actions on delete/update of the parent: **CASCADE, SET NULL, SET DEFAULT, RESTRICT/NO ACTION**.

MCQ Trap: Entity integrity = PK not NULL; Referential integrity = FK matches a PK or is NULL. A foreign key **can** be NULL but a **primary key can never** be NULL.

Memory Trick: A NULL is **not** zero and **not** an empty string - it means **"unknown / not applicable"**. NULL = NULL evaluates to **UNKNOWN**, not TRUE.

5. Relational Algebra and Calculus

5.1 Relational Algebra (Procedural)

Relational algebra is a **procedural** query language - it specifies **what** result and **how (the sequence of operations)**. Operators take **one or two relations** and return a **new relation** (closure property).

5.1.1 Unary Operations

- Selection (sigma):** picks **rows/tuples** that satisfy a predicate. Notation $\sigma_{\text{condition}}(R)$. It is **horizontal** partitioning. Example: $\sigma_{\text{age} > 20}(\text{Student})$.
- Projection (pi):** picks **columns/attributes** and **removes duplicate** rows. Notation $\pi_{\text{col-list}}(R)$. It is **vertical** partitioning.
- Rename (rho):** renames a relation/attributes. Notation $\rho_{\text{S}}(R)$.

MCQ Trap: Selection (sigma) = rows; Projection (pi) = columns. Projection **eliminates duplicates** (result is a set); selection does not change columns. Examiners swap sigma and pi.

5.1.2 Set Operations

Require **union-compatible** relations (**same number of attributes + same domains**).

Operation	Symbol (ASCII)	Result
Union	$R \cup S$	Tuples in R or S (no duplicates)
Intersection	$R \cap S$	Tuples in both R and S
Set Difference	$R - S$	Tuples in R but not in S
Cartesian Product	$R \times S$	Every tuple of R paired with every tuple of S

- For $R \times S$: degree = **degree(R)+degree(S)**; cardinality = **$|R| \times |S|$** .
- Union, Intersection are commutative; Difference is NOT** commutative.

5.1.3 Join Operations

- Theta Join ($R \bowtie_{\text{theta}} S$):** Cartesian product **followed by a selection** on condition theta (any comparison **<, >, =, ...**).
- Equi Join:** a theta join where theta uses **only equality (=)**.

- **Natural Join ($R \bowtie X S$):** equi-join on **all common attributes**, then **removes the duplicate** column. If no common attribute, it becomes a **Cartesian product**.
- **Outer Joins:** keep unmatched tuples, padding with **NULL**:
- **Left Outer Join** - keeps **all left** tuples.
- **Right Outer Join** - keeps **all right** tuples.
- **Full Outer Join** - keeps **all** tuples from both.
- **Semi Join ($R \ltimes X S$):** tuples of R that **have a match** in S (returns only R's columns).

R	S	R NATURAL JOIN S
+-----+	+-----+	+-----+
A B	B C	A B C
+-----+	+-----+	+-----+
a1 b1	b1 c1	a1 b1 c1
a2 b2	b3 c3	+-----+
+-----+	+-----+	(only matching B=b1 survives)

5.1.4 Division Operation

- **Division ($R \div S$):** returns tuples of R that are associated with **all** tuples of S. Used for **"for all" / "every"** queries (e.g. students who took **all** courses).

5.1.5 Aggregate Functions in Algebra

- Notation: $_(\text{grouping}) \ G \ _(\text{function}(\text{attr}))(R)$ using the **script G (gamma)** operator.
Functions: **SUM, AVG, COUNT, MIN, MAX**.

5.1.6 Derived vs Fundamental Operators

- **6 fundamental (primitive) operators:** Selection, Projection, Union, Set-Difference, Cartesian-Product, Rename.
- **Derived:** Intersection, all Joins, Division ($R \cap S = R - (R - S)$).

Memory Trick: Fundamental set = **"S P U D C R"** (Select, Project, Union, Difference, Cross, Rename). Intersection and joins can be built from these.

5.2 Relational Calculus (Non-Procedural)

Relational calculus is **non-procedural / declarative** - it states **what** is wanted, **not how**. Based on **first-order predicate logic**.

5.2.1 Tuple Relational Calculus (TRC)

- Variables range over **tuples**. Form: $\{ t \mid P(t) \}$ - "the set of tuples t such that predicate P(t) is true".
- Uses quantifiers **EXISTS (there-exists)** and **FOR ALL**.
- Example: $\{ t \mid t \text{ IN Student AND } t.\text{age} > 20 \}$.

5.2.2 Domain Relational Calculus (DRC)

- Variables range over **domain values (attributes)**, not whole tuples. Form: $\{ \langle x_1, x_2, \dots \rangle \mid P(x_1, x_2, \dots) \}$.
- TRC and DRC are **equivalent in expressive power** to (safe) relational algebra.

5.2.3 Safe Expressions

- An expression is **safe** if it produces a **finite** result and uses only values from the **domain of the database** (the active domain). Unsafe example: $\{ t \mid \text{NOT}(t \text{ IN } R) \}$ is infinite.

Aspect	Relational Algebra	Relational Calculus
Nature	Procedural (how)	Non-procedural (what)
Specifies	Sequence of operations	Only the desired result

Aspect	Relational Algebra	Relational Calculus
Variables	Relations	Tuples (TRC) / Domains (DRC)
Basis	Set theory / operators	Predicate logic
Equivalence	Same power as safe calculus	Same power as algebra
Used by	Query optimizer (internal)	High-level languages (SQL is closer to calculus)

MCQ Trap: Algebra = procedural (HOW); Calculus = declarative (WHAT). TRC variables = tuples, DRC variables = domain values. Codd's theorem: algebra and safe calculus are **equally expressive (relationally complete)**.

6. Structured Query Language (SQL)

SQL (originally **SEQUEL**, by **IBM** for **System R, 1974**) is the standard **declarative** language for relational DBs. First ANSI standard **SQL-86**; major: **SQL-92, SQL:1999, SQL:2003**.

6.1 SQL Basics

6.1.1 Characteristics

- **Non-procedural / declarative**, English-like, set-oriented.
- **Case-insensitive** for keywords; data is case-sensitive.
- SQL is based on **bag (multiset) semantics** by default - it **keeps duplicates** unless **DISTINCT** is used (this differs from relational algebra projection which removes duplicates).

6.1.2 SQL Data Types

Category	Types
Numeric	INT, SMALLINT, BIGINT, DECIMAL(p,s), NUMERIC, FLOAT, REAL
Character	CHAR(n) (fixed), VARCHAR(n) (variable), TEXT
Date/Time	DATE, TIME, TIMESTAMP, INTERVAL
Others	BOOLEAN, BLOB, CLOB

MCQ Trap: CHAR(n) is **fixed-length** (padded with blanks); **VARCHAR(n)** is **variable-length** (stores only what is entered). **CHAR** wastes space but is faster for fixed data.

6.2 DDL - Data Definition Language

```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(30) NOT NULL,
    Age INT CHECK (Age >= 17),
    DeptId INT REFERENCES Dept(DeptId),
    City VARCHAR(20) DEFAULT 'Delhi'
);
ALTER TABLE Student ADD Email VARCHAR(50);
ALTER TABLE Student MODIFY Name VARCHAR(40);
ALTER TABLE Student DROP COLUMN City;
TRUNCATE TABLE Student; -- removes all rows, keeps structure
DROP TABLE Student; -- removes table + structure
RENAME Student TO Learner;
```

Command	Effect	Rollback?	Structure kept?
DROP	Deletes table + data + structure	No (DDL)	No
TRUNCATE	Deletes all rows , keeps structure	No (DDL, auto-commit)	Yes
DELETE	Deletes selected rows (DML)	Yes	Yes

MCQ Trap: TRUNCATE is **DDL** (fast, no WHERE, no rollback, resets identity); DELETE is **DML** (slow, supports WHERE, **logged**, can ROLLBACK, fires triggers). DROP removes the whole table.

6.3 DML - Data Manipulation Language

```
INSERT INTO Student (RollNo, Name) VALUES (101, 'Ram');
UPDATE Student SET Age = 21 WHERE RollNo = 101;
DELETE FROM Student WHERE Age < 17;
```

6.4 DQL - Querying Data (SELECT)

The **logical order of clauses**: SELECT - FROM - WHERE - GROUP BY - HAVING - ORDER BY. The **execution (evaluation) order**: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

```
SELECT Branch, COUNT(*) AS Total, AVG(Age) AS AvgAge
FROM Student
WHERE Age >= 18
GROUP BY Branch
HAVING COUNT(*) > 2
ORDER BY AvgAge DESC;
```

- **WHERE** filters **rows before grouping**; **HAVING** filters **groups after aggregation**.
- Operators: comparison =, <, !=, <, >, <=, >=; logical AND, OR, NOT; range BETWEEN; set IN; pattern LIKE (% = any string, _ = single char); null test IS NULL / IS NOT NULL; **DISTINCT** removes duplicates.

Aggregate	Returns	Counts NULL?
COUNT(*)	Total rows	Yes
COUNT(col)	Non-null values	No
SUM, AVG	Sum / average	Ignores NULL
MIN, MAX	Smallest / largest	Ignores NULL

MCQ Trap: Aggregate functions **ignore NULLs** except COUNT(*). WHERE cannot use aggregate functions - aggregates belong in HAVING. AVG(col) divides by the count of **non-null** values only.

6.5 Joins in SQL

Join	Keeps
INNER JOIN	Only matching rows from both
LEFT [OUTER] JOIN	All left + matching right (NULLs for unmatched)
RIGHT [OUTER] JOIN	All right + matching left
FULL [OUTER] JOIN	All rows from both sides
CROSS JOIN	Cartesian product (no condition)
SELF JOIN	Table joined with itself (aliases)

```
SELECT s.Name, d.DeptName
FROM Student s INNER JOIN Dept d ON s.DeptId = d.DeptId;
```

Memory Trick: INNER = intersection, FULL OUTER = union of the two row-sets. A CROSS JOIN of m and n rows yields **m*n** rows.

6.6 Advanced Queries

6.6.1 Subqueries (Nested Queries)

- **Single-row subquery:** returns one value; use =, <, >.
- **Multi-row subquery:** returns many values; use IN, ANY, ALL.

- **Correlated subquery:** inner query **references the outer** query and runs **once per outer row** (slower).

```
-- non-correlated
SELECT Name FROM Student
WHERE Age > (SELECT AVG(Age) FROM Student);

-- correlated (uses EXISTS)
SELECT Name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enroll e WHERE e.RollNo = s.RollNo);
```

- **EXISTS** returns TRUE if the subquery returns **at least one row**; **NOT EXISTS** for the opposite. **EXISTS** often outperforms **IN** for large data.

6.6.2 Set Operators

Operator	Result	Duplicates
UNION	Combine, remove duplicates	Removed
UNION ALL	Combine, keep duplicates	Kept (faster)
INTERSECT	Common rows	Removed
MINUS / EXCEPT	In first, not in second	Removed

Set operators require the SELECTs to be **union-compatible** (same column count + compatible types).

6.6.3 Views

- A **view** is a **virtual table** based on a stored query; it stores **no data** (except a **materialized view**, which does).
- **Advantages:** security (hide columns), simplicity, logical data independence.
- A view is **updatable** only if it maps to **one base table**, includes its PK, and has **no aggregates, DISTINCT, GROUP BY, or joins**.

```
CREATE VIEW CSE_Students AS
SELECT RollNo, Name FROM Student WHERE Branch = 'CSE';
```

6.6.4 Indexes

- An **index** speeds up retrieval (like a book index). Costs extra storage and slows **INSERT/UPDATE/DELETE**.
- **Clustered index:** determines the **physical order** of rows; **only one per table** (often the PK).
- **Non-clustered index:** a separate structure with pointers; **many allowed** per table.

MCQ Trap: Only **one clustered index** per table (data can be sorted one way only); but **many non-clustered** indexes. A **PRIMARY KEY** usually creates a **clustered** index by default.

6.6.5 Sequences

- A **sequence** generates **unique numeric values** (e.g. for surrogate keys): **CREATE SEQUENCE s START WITH 1 INCREMENT BY 1**; then **s.NEXTVAL**, **s.CURRVAL**.

6.7 DCL and TCL

```
-- DCL
GRANT SELECT, INSERT ON Student TO user1;
REVOKE INSERT ON Student FROM user1;

-- TCL
SAVEPOINT sp1;
UPDATE Student SET Age = 22 WHERE RollNo = 101;
ROLLBACK TO sp1;    -- undo to savepoint
COMMIT;             -- make changes permanent
```


- **GRANT** gives privileges; **REVOKE** removes them. **WITH GRANT OPTION** lets the grantee re-grant.
- **COMMIT** = permanent; **ROLLBACK** = undo to last commit or savepoint; **SAVEPOINT** = a marker for partial rollback.

6.8 PL/SQL and Active Database

- **PL/SQL** = Procedural Language extension to SQL (variables, loops, IF, exceptions) by **Oracle**.
- **Stored Procedure**: named precompiled block; can return many/no values via **OUT** params; invoked with **CALL/EXEC**.
- **Function**: **must return exactly one value**; usable inside SQL expressions.
- **Trigger**: a block that fires **automatically** on **INSERT/UPDATE/DELETE**; timing **BEFORE/AFTER**, level **ROW/STATEMENT**. Used to enforce complex rules/auditing.
- **Cursor**: a pointer to the result set, processed **row by row**. **Implicit** (auto) vs **Explicit** (declared: **OPEN, FETCH, CLOSE**).
- **Assertion**: a named **constraint on the whole database** (any condition that must always hold).

Concept	Returns	Called automatically?
Procedure	0..many (OUT)	No (explicit CALL)
Function	Exactly 1	No (used in expressions)
Trigger	None	Yes (on DML event)

MCQ Trap: A **function returns exactly one value** and can be used in a **SELECT**; a **trigger fires automatically** and cannot be called manually. A **cursor** processes results **one row at a time**.

7. Relational Database Design and Normalization

Normalization = a step-by-step process of decomposing relations to **reduce redundancy** and eliminate **insertion, deletion and update anomalies**, while preserving information.

7.1 Functional Dependencies (FD)

- An **FD** $X \rightarrow Y$ ("X determines Y") means: any two tuples with the same X value **must** have the same Y value. **X = determinant**, Y = dependent.
- **Trivial FD**: $X \rightarrow Y$ where **Y is a subset of X** (always holds, e.g. $AB \rightarrow A$).
- **Non-trivial FD**: $X \rightarrow Y$ where Y is **not** a subset of X.

7.1.1 Armstrong's Axioms (Sound and Complete)

Axiom	Rule
Reflexivity	If Y is subset of X, then $X \rightarrow Y$
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Derived rules:

Rule	Statement
Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
Pseudo-transitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
Composition	If $X \rightarrow Y$ and $Z \rightarrow W$, then $XZ \rightarrow YW$

Memory Trick: The 3 primary axioms are "**RAT**" = **R**eflexivity, **A**ugmentation, **T**ransitivity. They are **sound** (derive only valid FDs) and **complete** (derive all valid FDs).

7.1.2 Attribute Closure (X^+)

- **X^+** = set of all attributes **functionally determined by X**. Algorithm: start with X, repeatedly add the right side of any FD whose left side is already in the set, until no change.
- **Uses of closure:** (1) check if X is a **super key** (if $X^+ =$ all attributes); (2) test if an FD $X \rightarrow Y$ holds (if Y is subset of X^+); (3) find candidate keys.

7.1.3 Canonical / Minimal Cover

A **minimal cover (F_{min})** is an equivalent FD set such that: (1) every RHS has a **single attribute**; (2) **no redundant FD**; (3) **no redundant attribute** on any LHS (extraneous attributes removed).

7.1.4 Equivalence of FD Sets

- F and G are **equivalent ($F \equiv G$)** iff $F^+ = G^+$, i.e. **F covers G AND G covers F** (each FD of one is derivable from the other).

7.2 Finding Candidate Keys from FDs

- Attributes appearing **only on the LHS** (never on RHS) must be in **every** candidate key.
- Attributes appearing **only on the RHS** are **never** in a candidate key.
- Attributes **not in any FD** must be in every candidate key.
- Compute closures of LHS-essential attribute combinations; if a set's closure = all attributes and it is minimal, it is a **candidate key**.

7.2.1 Prime vs Non-Prime Attributes

- **Prime attribute:** part of **some** candidate key.
- **Non-prime attribute:** part of **no** candidate key.

7.3 Decomposition

- **Lossless (non-loss) join decomposition:** the natural join of the parts gives back **exactly** the original relation (no spurious tuples). For a binary split R into R1, R2 it is lossless iff **$(R1 \cap R2) \rightarrow R1$ or $(R1 \cap R2) \rightarrow R2$** (the common attributes form a key of at least one part).
- **Dependency-preserving decomposition:** the union of FDs of the parts is **equivalent** to the original FD set, so all constraints can be checked **without a join**.

MCQ Trap: Lossless join must ALWAYS be guaranteed; dependency preservation is desirable but **not always achievable in BCNF**. A decomposition can be lossless yet **not** dependency-preserving.

7.4 Normal Forms

7.4.1 Anomalies

- **Insertion anomaly:** cannot insert a fact without other (possibly unknown) data.
- **Deletion anomaly:** deleting a tuple unintentionally loses other facts.
- **Update anomaly:** redundant data must be changed in **many** places; missing one causes inconsistency.

7.4.2 The Normal Forms

NF	Condition (eliminates)
1NF	All attributes atomic (no multivalued/composite/repeating groups)
2NF	1NF + no partial dependency (no non-prime attr depends on part of a candidate key)
3NF	2NF + no transitive dependency of a non-prime attribute on a key
BCNF	For every non-trivial $X \rightarrow Y$, X is a super key
4NF	BCNF + no non-trivial multivalued dependency (MVD)
5NF (PJNF)	4NF + no non-trivial join dependency (every JD is implied by candidate keys)

- **3NF formal rule:** for every FD $X \rightarrow A$, **either X is a super key, OR A is a prime attribute.**
- **BCNF** is stricter: it drops the "A is prime" escape - **X must be a super key** for every non-trivial FD.

1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF $\rightarrow$ 4NF $\rightarrow$ 5NF
 (atomic) (no partial) (no transitive) (X is superkey) (no MVD) (no join dependency)
 weakest -----> strongest

NF	Removes	Based on
2NF	Partial dependency	FD
3NF	Transitive dependency	FD
BCNF	All FD-based anomalies	FD
4NF	Multivalued dependency	MVD
5NF	Join dependency	JD

MCQ Trap: 2NF removes **PARTIAL** dependency; 3NF removes **TRANSITIVE** dependency; **BCNF:** every determinant is a super key. A relation with **all attributes prime** and a **single-attribute key** is automatically in 3NF/BCNF.

MCQ Trap: 3NF is always achievable with both lossless join AND dependency preservation. **BCNF guarantees lossless join but may SACRIFICE dependency preservation.** This is the single most-tested normalization fact.

Memory Trick: 2NF/3NF/BCNF deal with **FDs**; 4NF deals with **MVDs**; 5NF deals with **JDs**. Order of strictness: "**1 < 2 < 3 < BCNF < 4 < 5**".

7.4.3 Multivalued Dependency (MVD) and Join Dependency (JD)

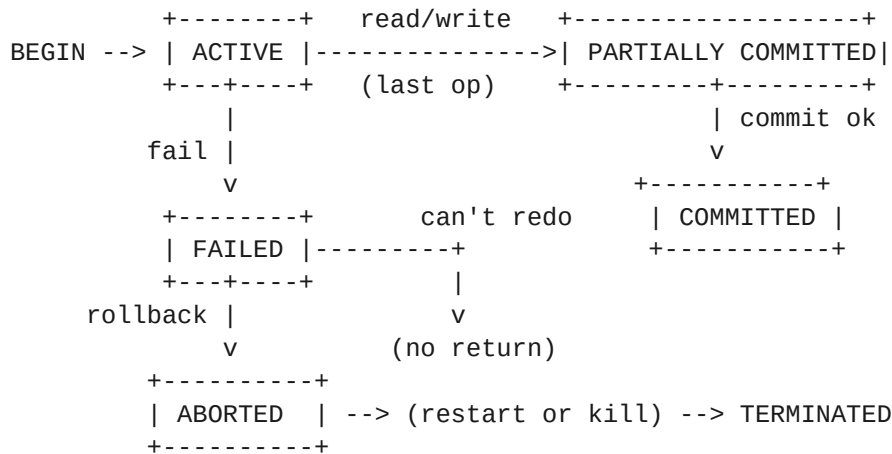
- **MVD $X \twoheadrightarrow Y$:** for each X, a **set** of Y values exists independent of the other attributes (causes 4NF violation). Every FD is an MVD but not vice versa.
- **Join Dependency:** a relation can be **losslessly decomposed into 3 or more** relations; absence of a non-trivial JD gives **5NF**.

8. Transaction Management and Concurrency Control

8.1 Transaction Concepts

- A **transaction** is a **logical unit of work** - a sequence of operations executed as a **single all-or-nothing** unit. Bracketed by **BEGIN ... COMMIT/ROLLBACK**.
- Primitive operations: **read(X)** and **write(X)**.

8.1.1 Transaction States



The **5 states**: **Active**, **Partially Committed**, **Committed**, **Failed**, **Aborted** (plus Terminated). Once **Committed**, effects are **permanent** and cannot be undone.

8.1.2 ACID Properties

Property	Meaning	Ensured by
Atomicity	All-or-nothing (no partial)	Transaction (recovery) manager
Consistency	DB moves from one valid state to another	Application + constraints
Isolation	Concurrent txns do not interfere	Concurrency-control manager
Durability	Committed changes survive crashes	Recovery manager (logs)

Memory Trick: ACID = **A**tomicity (all/nothing), **C**onsistency (rules hold), **I**solation (no interference), **D**urability (survives crash). Atomicity + Durability are handled by **recovery**; Isolation by **concurrency control**.

MCQ Trap: Atomicity is about "all-or-nothing"; **Durability** is about "survives failure after commit". Examiners swap these. Isolation gives the illusion of **serial** execution.

8.2 Schedules

- A **schedule** is the **chronological order** of operations from one or more concurrent transactions.
- Serial schedule**: transactions run **one after another** with no interleaving - always **correct** but no concurrency.
- Non-serial schedule**: operations are **interleaved** - gives concurrency but may be incorrect unless **serializable**.

8.2.1 Serializability

A non-serial schedule is **serializable** if it is **equivalent to some serial schedule** (same final effect).

- Conflict Serializability**: schedule can be transformed into a serial one by **swapping non-conflicting** operations. Two ops **conflict** if they are from **different transactions**, on the **same data item**, and **at least one is a WRITE** (R-W, W-R, W-W conflict; R-R does not).
- Test using a **precedence (serialization) graph**: nodes = transactions, edge $T_i \rightarrow T_j$ if T_i has an op conflicting and earlier than T_j 's. The schedule is **conflict serializable iff the graph is ACYCLIC**.
- View Serializability**: weaker/more general; equivalent on **initial reads, write-read (read-from) pairs, and final writes**. **Every conflict-serializable schedule is view-serializable, but not vice versa** (the extra ones contain **blind writes**).

Precedence graph (3 txns):
 (T1) ---> (T2)
 ^ |

$\begin{array}{c} | \\ +----- \end{array} \quad \begin{array}{c} \vee \\ (T3) \end{array} \quad \leftarrow \text{CYCLE present} \Rightarrow \text{NOT conflict serializable}$

MCQ Trap: Conflict-serializable (subset of) View-serializable. A cycle in the precedence graph means **NOT conflict serializable**. R-R operations **never conflict**. A **blind write** (write without a prior read) is what makes a schedule view- but not conflict-serializable.

8.2.2 Recoverability of Schedules

Schedule type	Condition
Recoverable	Tj commits after Ti, if Tj read a value written by Ti
Cascadeless (ACR)	A txn reads only values written by committed txns (no dirty reads)
Strict	No txn may read OR write a value until the writer commits/aborts

Strictness order: **Strict (subset of) Cascadeless (subset of) Recoverable**. **Cascading rollback** (one abort forces others) is avoided in cascadeless schedules.

MCQ Trap: Strict $\Rightarrow$ Cascadeless $\Rightarrow$ Recoverable (strict is the most restrictive). A non-recoverable schedule can lose durability and must be avoided.

8.3 Concurrency Control Protocols

8.3.1 Problems of Concurrency

Problem	Cause
Lost Update	Two txns write the same item; one overwrites the other (W-W)
Dirty Read (uncommitted dependency)	Reading a value written by an uncommitted txn (W-R)
Unrepeatable Read	Re-reading an item gives a different value (R-W)
Phantom Read	New rows appear/disappear between two reads of a range

8.3.2 Lock-Based Protocols

- Shared lock (S):** for **reading**; multiple txns may hold S on the same item.
- Exclusive lock (X):** for **writing**; only **one** txn may hold it; incompatible with all others.

Lock compatibility:

	S	X
S	YES	NO
X	NO	NO

8.3.3 Two-Phase Locking (2PL)

Guarantees **conflict serializability**. Each transaction has:

- Growing phase:** can **acquire** locks, **cannot release** any.
- Shrinking phase:** can **release** locks, **cannot acquire** any.

The point of the last lock acquired is the **lock point**.

2PL variant	Releases locks
Basic 2PL	Any time in shrinking phase
Strict 2PL	Holds all exclusive (X) locks until commit/abort
Rigorous 2PL	Holds ALL (S and X) locks until commit/abort
Conservative (Static) 2PL	Acquires all locks before starting; deadlock-free

MCQ Trap: 2PL guarantees conflict serializability but NOT deadlock-freedom (except Conservative 2PL). **Strict/Rigorous 2PL** also guarantee **cascadeless + strict** (recoverable) schedules. **Conservative 2PL is the only deadlock-free** variant but may cause starvation.

8.3.4 Deadlock and Starvation

- **Deadlock**: a cycle of transactions each waiting for a lock held by another. Handled by: **deadlock prevention** (Wait-Die, Wound-Wait using timestamps), **deadlock detection** (wait-for graph + victim selection), or **timeouts**.
- **Wait-Die (non-preemptive)**: older waits, younger dies (restarts).
- **Wound-Wait (preemptive)**: older wounds (aborts younger), younger waits.
- **Starvation**: a transaction waits indefinitely; solved by **aging / FIFO** lock grants.

Memory Trick: "Wait-Die: old waits; Wound-Wait: old wounds." In both, the **older** transaction is favored (never restarts indefinitely), preventing starvation.

8.3.5 Timestamp-Based Protocol

- Each txn gets a unique **timestamp TS(T)** at start (older = smaller). Each item X has **read-TS (R-TS)** and **write-TS (W-TS)**.
- A read/write that would **violate timestamp order** (i.e. arrives "too late") causes the txn to **abort and restart** with a new timestamp.
- **No locks => no deadlock**, but possible **starvation** (repeated restarts) and **cascading rollbacks**.

8.3.6 Thomas' Write Rule

- An **optimization** of timestamp ordering: an **outdated (obsolete) write** is simply **ignored** rather than causing an abort. It allows some **view-serializable** (non-conflict-serializable) schedules.

8.3.7 Validation / Optimistic Concurrency Control (OCC)

- Assumes conflicts are **rare**; runs in **3 phases**: **Read phase -> Validation phase -> Write phase**. Checks for conflicts only at validation; good for **read-heavy** workloads.

8.3.8 Multiple Granularity Locking

- Locks at different levels (**DB -> file -> page -> record**) arranged as a **tree**; uses **intention locks (IS, IX, SIX)** on ancestors before locking a node, improving concurrency.

Protocol	Deadlock?	Guarantees
Basic 2PL	Possible	Conflict serializability
Conservative 2PL	No	Serializability (deadlock-free)
Timestamp ordering	No	Serializability (may starve)
Optimistic (OCC)	No	Serializability (validation)

MCQ Trap: Lock-based protocols can deadlock; timestamp/optimistic protocols cannot deadlock (they abort/restart instead). **Thomas' Write Rule** ignores obsolete writes and permits some view-serializable schedules.

9. Recovery and File / Storage Structure

9.1 Database Recovery

9.1.1 Failure Classification

- **Transaction failure** (logical error or system-enforced abort, e.g. deadlock victim).
- **System crash** (power/hardware/software failure - main memory lost, disk intact).
- **Disk / media failure** (head crash - non-volatile storage lost; needs backups).

9.1.2 Log-Based Recovery

The **log (journal)** is an append-only sequence of records on **stable storage**. A typical record: **<Ti, X, old_value, new_value>**.

- **Deferred update (NO-UNDO/REDO):** writes are **deferred to disk until commit**; on recovery only **REDO** committed txns. No undo needed.
- **Immediate update (UNDO/REDO):** writes may hit disk **before commit**; on recovery **UNDO** uncommitted and **REDO** committed txns.
- **Write-Ahead Logging (WAL):** the **log record must reach disk BEFORE** the corresponding data page - the golden rule of recovery.

Technique	UNDO	REDO
Deferred update	No	Yes
Immediate update	Yes	Yes

9.1.3 Checkpoints

- A **checkpoint** periodically flushes log + buffers to disk and records a **<checkpoint>** marker, so recovery need only scan back to the **last checkpoint** instead of the entire log - **shortening recovery time**.

9.1.4 Shadow Paging

- Maintains **two page tables**: a **current** and a **shadow (old)**. Updates go to **new copies**; on commit the current table becomes shadow; on failure revert to shadow. **No log/UNDO/REDO needed**, but causes **data fragmentation** and is hard to use with concurrency.

9.1.5 ARIES (overview)

- A practical recovery algorithm using **3 passes: Analysis -> Redo -> Undo**, with **WAL, LSNs (Log Sequence Numbers)** and **fuzzy checkpoints**.

MCQ Trap: Deferred update = NO UNDO needed (REDO only); Immediate update = both UNDO + REDO. WAL says the log is written before the data page. Shadow paging needs no log.

9.2 File Organization and Indexing

9.2.1 Storage Hierarchy

Registers < Cache < Main Memory < Flash/SSD < Magnetic Disk < Tape
(fastest, costliest, volatile) ----> (slowest, cheapest, non-volatile)

- **Primary (volatile):** registers, cache, RAM. **Secondary (non-volatile):** disk, SSD. **Tertiary:** tape, optical.

9.2.2 File Organization

Organization	Idea	Best for
Heap (unordered)	Records placed anywhere	Fast insert; O(n) search
Sequential (ordered)	Sorted on a key	Range queries / ordered scan
Hash	Bucket = hash(key)	O(1) equality search; poor for range

9.2.3 Indexing

An **index** is an auxiliary (**search-key, pointer**) structure that speeds lookups.

Index	Description
Primary (clustering) index	On the ordering key of a sequential file; data is sorted by it
Secondary index	On a non-ordering field; must be dense
Dense index	One index entry for every record
Sparse index	One entry per block/page (only on sorted data)
Multilevel index	Index built on top of an index (reduces I/O), leads to B+ trees

MCQ Trap: A **primary index can be sparse**; a **secondary index must be dense**. There is **at most one primary/clustering index** per file but **many secondary** indexes. "Primary index" is built on the ordering key, not necessarily the primary key.

9.2.4 B-Tree and B+ Tree

A **B-tree of order m** is a balanced multi-way search tree:

- Every node has at most **m children** and at most **m-1 keys**.
- Non-root internal nodes have at least **ceil(m/2) children**.
- All **leaves at the same level** (perfectly height-balanced).
- Search / insert / delete: **O(log n)** (logarithm base = order, so very shallow -> few disk I/Os).

B+ TREE (order 3): keys only in leaves; leaves linked
 [13 | 30] <- internal (index) nodes
 / | \
 [5|11] [13|24] [30|41] <- leaf nodes hold data ptrs
 |->-----|->-----| <- leaves linked list (range scan)

Feature	B-Tree	B+ Tree
Data pointers	In all nodes	Only in leaf nodes
Leaf linking	No	Yes (linked list)
Range query	Slower	Faster (sequential leaves)
Same key copies	Single copy	Key may repeat in internal + leaf
Tree height / fan-out	Higher	Lower (more keys per internal node)

MCQ Trap: In a **B+ tree all actual data/record pointers are in the LEAVES**, and **leaves are linked** for efficient range scans; **B-tree stores data in internal nodes too**. B+ tree is the default index in most RDBMS. Both keep all leaves at the **same level** and give **O(log n)** operations.

Memory Trick: "**B+ = data at the Bottom (leaves) + linked.**" This is why range queries are faster in B+ trees.

9.2.5 Hashing

- **Hashing** maps a key to a **bucket address** via a **hash function h(key)**; ideal lookup is **O(1)**.
- **Collision:** two keys map to the same bucket; resolved by **chaining / open addressing / overflow buckets**.

Type	Behavior
Static hashing	Fixed number of buckets; degrades as data grows (overflow chains)
Dynamic hashing	Bucket count grows/shrinks with data
Extendible hashing	Uses a directory + global/local depth ; splits one bucket at a time (doubles directory)
Linear hashing	Splits buckets in round-robin order; no directory needed

MCQ Trap: **Static hashing suffers from overflow** when the file grows; **extendible (dynamic) hashing** adapts by **doubling the directory** and uses **global vs local depth**. Hashing gives **O(1) equality search** but is **poor for range queries** (use B+ trees for ranges).

10. Advanced and Emerging Topics (Overview)

10.1 Distributed Databases

- Data stored across **multiple physical sites** connected by a network, appearing as one logical DB.
- **Transparencies:** **location, fragmentation, replication** transparency.

- **Fragmentation: horizontal** (rows), **vertical** (columns), **hybrid/mixed**.
- **CAP theorem (Brewer):** a distributed system can guarantee at most **2 of 3: Consistency, Availability, Partition-tolerance**.

10.2 Data Warehousing and OLAP

- A **data warehouse** is a **subject-oriented, integrated, time-variant, non-volatile** collection for analysis (Bill Inmon's definition).
- **OLTP** (transactional, day-to-day, normalized, many short writes) vs **OLAP** (analytical, historical, denormalized, complex reads).
- Schemas: **Star, Snowflake, Fact-Constellation**. Operations: **Roll-up, Drill-down, Slice, Dice, Pivot**. **ETL** = Extract-Transform-Load.

Feature	OLTP	OLAP
Purpose	Daily transactions	Decision support / analysis
Operations	Short read/write	Complex read-heavy queries
Design	Normalized (3NF)	Denormalized (star)
Data	Current	Historical / aggregated

10.3 NoSQL Databases

- **"Not Only SQL"** - schema-less, horizontally scalable stores for big/unstructured data. **4 types:**
- **Key-Value** (Redis, DynamoDB), **Document** (MongoDB, CouchDB), **Column-family** (Cassandra, HBase), **Graph** (Neo4j).
- Often follow **BASE** (**B**asically **A**vailable, **S**oft state, **E**ventual consistency) instead of ACID.

10.4 Query Processing and Optimization

- Steps: **Parsing & translation -> Optimization -> Evaluation**.
- Optimizer picks the cheapest **query execution plan** using **cost estimation** (I/O, CPU) and **heuristics** (e.g. push **selections/projections down** the tree first to shrink intermediate results).

MCQ Trap: OLTP = normalized, write-heavy, current data; OLAP = denormalized star schema, read-heavy, historical. CAP theorem allows only 2 of 3 guarantees. NoSQL favors BASE over ACID.

11. Quick Revision / Exam Pointers

- **Codd** gave the relational model (**1970**); **Chen** gave ER (**1976**); **ANSI/SPARC** 3-schema (**1975**).
- **Degree = columns, Cardinality = rows**. Primary key = NOT NULL + UNIQUE.
- **Selection = rows (sigma), Projection = columns (pi)**. 6 fundamental algebra ops.
- **2NF = partial, 3NF = transitive, BCNF = determinant is super key**. 3NF keeps dependency preservation; BCNF may not.
- **ACID**; conflict serializable iff **precedence graph acyclic**; conflict (subset of) view serializable.
- **2PL = conflict serializable, not deadlock-free; Conservative 2PL = deadlock-free**; timestamp/OCC abort instead of deadlock.
- **Deferred = REDO only; Immediate = UNDO+REDO; WAL = log before data**.
- **B+ tree: data in leaves, leaves linked, O(log n)**. Hashing: **O(1)** equality, bad for ranges.
- Practice: find candidate keys via **attribute closure**, identify **highest normal form**, decompose losslessly, draw **precedence graphs**, and predict **SQL output** (JOIN/GROUP BY/HAVING/NULL behavior).